

Synchronization Architecture & Scenarios

Operational design for unreliable networks, direct device administration, multi-device conflicts, durable synchronization, and recovery.

DESIGN BRIEF • 2026

› Core principle: device usability must survive backend/network failure; synchronization must be durable, resumable, auditable, and safe against incomplete reads.

Executive Snapshot

This document defines the expected synchronization behavior between the React admin UI, Spring Boot backend, Windows Gateway, and TrueFace readers. It focuses on the failure modes and operating conditions observed in the latest gateway/backend material and the supplied three-day logs.

2	1,196	11001	10054
assigned devices	users to synchronize	DNS failure code	WebSocket reset code

› Safety invariant: a partial or empty roster is a failed/incomplete read—not evidence that users should be deleted.

Gym Device ↔ Gateway ↔ Backend Synchronization Architecture, Scenarios & Recovery Rules
Production design document based on the latest Gateway/Backend source and the 2–4 October 2026 gateway logs

1. Purpose

This document defines the synchronization contract for a gym installation with unreliable Internet, direct administrator edits on TrueFace devices, one or more devices assigned to a Windows Gateway, and a Java/Spring Boot backend. It covers normal flow, offline operation, reconciliation, photos, conflicts, recovery, and the issues observed in the supplied three-day logs.

2. System Context

3. Provisioning / Effective Configuration

- Admin creates the gym in React.
- Admin creates a Gateway and enrollment token.
- Admin creates two devices and assigns them to that Gateway.
- Windows MSI is installed.
- Gateway Configurator receives gateway ID/enrollment credential and refreshes assigned devices.
- Operator saves and starts configuration.
- Gateway loads the persisted ProgramData configuration and connects to the assigned devices and backend.
- Important: appsettings.json should contain application defaults. Production gateway identity, protected credentials and assigned device configuration come from the persisted Configurator configuration. Polling/retry defaults may remain application defaults unless explicitly made configurable.

4. Two Independent Synchronization Directions

The backend network must never be part of a TrueFace SDK critical section. Device work must be able to succeed while backend delivery is temporarily deferred.

5. Server → Device Normal Flow

React submits a business change.

Java persists the desired state and creates a gateway command with message/correlation identifiers.

Gateway receives it over WebSocket; REST is fallback.

Gateway places it in the target device queue.

Only the device executor touches that reader's SDK.

Gateway executes CREATE/UPDATE/DELETE/ENABLE/DISABLE/VALIDITY or equivalent.

Device execution result is recorded separately from backend result delivery.

If delivery of the result fails, the result is retained durably and retried.

6. Device → Server Normal Flow

Gateway receives a reliable device event or discovers a difference during reconciliation.

Gateway performs a targeted GetUser/GetFace when possible.

Gateway compares the snapshot with local known state.

Only actual changes are emitted.

Outbound change/photo state is persisted durably before it is considered safely queued.

WebSocket is preferred; REST is fallback.

Failed delivery remains pending and is retried with backoff.

Backend processes messages idempotently using message/correlation IDs.

7. 1,195/1,196 User + Photo Synchronization

A batch of 20 or 40 is NOT a limit of 20 or 40 users. The complete roster must be processed. Batching only controls how many users are handled per SDK/HTTP operation.

- Example: 1,196 users → batches 1–20, 21–40, ..., final partial batch.
 - All 1,196 users remain eligible for synchronization.
 - A failed photo is retried independently according to the retry policy.
 - An incomplete roster is never interpreted as mass deletion.
 - Face metadata/hash should be compared first when the SDK supports it.
 - Only changed photos should be downloaded/uploaded.
 - Photo transfer must be resumable after network loss or gateway restart.
-

8. Source-of-Truth Rules

9. Direct Administrator Edits on Device

10. When Should the Device Update the Server?

- Immediately/near-real-time after a reliable device event.
 - After a targeted verification triggered by an event.
 - During periodic roster reconciliation when a difference is detected.
 - During periodic face reconciliation when a face version/hash differs.
 - After backend recovery, drain durable changes and pending photos.
 - Periodic full scans are safety mechanisms, not the primary real-time path.
-

11. Both Devices Change the Same User

Two devices may independently change the same logical user. The backend must not use message arrival time as the only conflict rule.

Recommended metadata: logical user ID, device ID, source, sourceChangedAt, messageId, correlationId, serverVersion, faceHash/faceVersion, processedAt, status.

12. Network / Backend Failure Scenarios

13. Actual Issues Observed in the Last Three Days

14. Device Queue and Priority

- Serialize SDK access per device, but do not hold the lock across HTTP.
 - Yield between face batches so high-priority commands can run.
 - Do not globally block reader B while reader A performs a large face sweep.
-

15. Photo Synchronization Contract

Read the complete user roster.

Determine face metadata/hash when available.

Compare against known backend/gateway face version.

Read JPEG only when a change is required.

Persist the face/upload state locally.

Upload in bounded batches or controlled concurrency.

Backend acknowledges each photo independently.

Retry only failed/incomplete items.

Do not redownload unchanged photos on every sweep.

16. Recovery Scenarios

17. Logging / Observability Contract

Every sync operation should include gatewayId, deviceId, deviceId where applicable, messageId, correlationId, operation, source, attempt, durationMs and result.

- ROSTER_SYNC_START device=D1 expected=1196
 - ROSTER_SYNC_COMPLETE device=D1 users=1196 durationMs=...
 - FACE_BATCH_READ device=D1 batch=12 size=40 success=39 failed=1
 - FACE_UPLOAD_DEFERRED gateway=G1 device=D1 user=123 reason=DNS
 - DEVICE_CHANGE_QUEUED gateway=G1 device=D1 user=123 messageId=...
 - DEVICE_CHANGE_ACK gateway=G1 device=D1 user=123 messageId=...
 - COMMAND_EXECUTION_SUCCESS device=D1 user=123 command=CREATE_USER durationMs=...
 - COMMAND_RESULT_DELIVERY_DEFERRED correlationId=... reason=BACKEND_UNAVAILABLE
 - ROSTER_READ_INCOMPLETE device=D1 reported=800 previous=1196 action=NO_DELETE
-

18. Scenario Matrix

19. Connectivity State Machine

CONNECTED → DEGRADED → OFFLINE → RECONNECTING → CONNECTED

- CONNECTED: WebSocket healthy; REST fallback only.
 - DEGRADED: transient failures; monitor and reduce retries.
 - OFFLINE: stop high-frequency retry; continue durable local work.
 - RECONNECTING: exponential backoff with jitter.
 - CONNECTED again: drain durable queue in controlled batches.
-

20. Recommended Final Architecture

React/Admin → Spring Boot → Gateway command transport → per-device executor → TrueFace

TrueFace → Gateway change detector → durable outbox/face queue → Spring Boot → business database

- Backend transport is asynchronous and durable.
 - Device execution is serialized per reader and priority-aware.
 - Roster and face reconciliation are background work.
 - Direct device edits are first-class events.
 - Conflicts use explicit backend version/source rules.
 - All outbound changes are traceable.
 - Destructive deletion requires a trustworthy complete read or explicit administrative intent.
-

21. Implementation Priorities

22. Acceptance Criteria

- Internet can be unavailable for hours without losing device-originated changes.
- Gateway restart does not lose pending changes/photos.
- Partial/empty roster never causes mass deletion.
- All 1,196 users can be synchronized; batch size is not a user limit.
- One failed photo does not permanently block unrelated users.
- High-priority commands are not indefinitely blocked by face sweep.
- Two-device conflicts are deterministic and auditable.
- Logs distinguish SDK execution from backend delivery.
- No credentials or face bytes are logged.
- WebSocket/REST alternation does not destabilize the gateway.
- Roster persistence survives restart and handles concurrent saves safely.

23. Evidence / Notes

The design was derived from the latest uploaded Gateway/Backend material and the supplied gateway logs for 2–4 October 2026. The logs explicitly show two assigned devices, ProgramData configuration loading, approximately 1,195-user rosters, DNS 11001 failures, WebSocket 10054 resets, partial/empty roster reads, roster-file persistence failures, long reconciliation/ListUsers operations, face-upload deferrals, SDK timeouts, and command-result delivery ambiguity.

This document is intended to be the shared contract between the Java backend, C# Gateway and QA/operations teams. Any change to conflict rules, deletion semantics, batching, or retry behavior should update this document and its acceptance tests.

Component	Responsibility	Role
React/Admin UI	Creates gym, gateway, enrollment token, devices and device assignments.	Business/admin intent
Java/Spring Boot	Owns gateway/device/business state and receives gateway changes.	Server business state
Windows Gateway	Bridges backend and TrueFace; durable local state/outbox.	Edge synchronization
TrueFace device	Executes user/face/access operations and can be edited locally.	Device-local state
Gateway Configurator	Enrolls gateway and persists operational configuration.	Provisioning

Direction	Trigger	Flow
Server → Device	React/admin changes	Spring Boot → WebSocket/REST → Gateway command queue → per-device executor → TrueFace

Direction	Trigger	Flow
Device → Server	Device event or reconciliation	TrueFace → Gateway detector → durable outbox/face queue → Spring Boot

Situation	Authority / rule
Gateway identity and assigned devices	Provisioning/backend + persisted Configurator state
Server-created user	Backend business state; gateway applies desired state
Direct device edit	Device is source of the new observation; backend accepts it through explicit version/source policy
Pending outbound change	Gateway durable outbox
Incomplete roster	Previous known roster; do not infer deletions
Photo	Per-user face hash/version + upload state
Two devices change same user	Backend deterministic conflict/version policy; never arrival-order-only

Device action	Gateway	Backend
Name change	Read and report targeted user change.	Apply if newer/accepted; audit source=device.
Validity/access change	Read and report changed fields.	Apply under version policy.
Face change	Read face/metadata and upload only if changed.	Store new face version idempotently.
Local user creation	Detect new user and report complete snapshot.	Create/link server-side user.
Local deletion	Report only when deletion is trustworthy.	Apply deletion/tombstone policy.
Empty/partial roster	Treat as failed/incomplete.	No mass deletions.

Case	Recommended resolution
Device A changes name; Device B changes face	Merge non-overlapping fields where supported.
Both change same field	Use explicit server version/source timestamp policy; audit the losing change.
A was offline; B changed server	A reconnects with stale version; compare before overwrite.
Both changed before either reached server	Backend receives both durable events and resolves deterministically.
Stale device event arrives after newer server change	Reject/ignore stale update and return current version/reconcile.

Failure	Required behavior	Must not happen
DNS 11001	Mark backend unavailable; durable queue; exponential backoff.	Rapid retry storm or data loss.
WebSocket 10054/reset	Close session, reconnect, use REST fallback.	Crash or lose queued changes.

Failure	Required behavior	Must not happen
REST unavailable	Continue device work; retain outbound data.	Block SDK on HTTP.
HTTP 409 concurrent modification	Treat as version/conflict; reload/reconcile.	Blind infinite retry.
Internet down for hours	Device remains usable; queue grows durably; drain on recovery.	Drop changes.
Backend slow	Explicit transport timeout and retry.	Hold device lock for HTTP.

Issue observed	Evidence / impact	Design response
DNS failure	app.kainazi.com SocketException 11001; face uploads and device changes deferred.	Circuit/backoff + durable outbox.
WebSocket reset	SocketException 10054 / incomplete close handshake.	Connection state machine + reconnect.
Heartbeat failure	Heartbeat write failed after remote close.	Connection-owned heartbeat.
ChannelReader.Count exception	NotSupportedException broke AcceptCommand in WebSocket/REST paths.	Never call unsupported Count; maintain explicit queue counters.
Roster file access denied/file in use	RosterStateStore MoveFile/WriteAllText failures.	Single writer per device; atomic save; retry sharing violations.
Roster directory missing	DirectoryNotFoundException for ProgramData roster temp file.	Ensure directory before every save and at startup.
Partial roster	395/1195 and 895/1195 users disappeared in scans.	No deletion on incomplete read.
Empty roster	Reader returned zero users; treated as failed read.	Retry; no deletion.
Long ListUsers/Reconcile	ListUsers reached ~105 s; Reconcile ~96 s in logs.	Separate workloads; priority executor.
Slow SDK calls	GetUser/GetFace commonly ~5 s; CreateUser up to ~15 s.	Targeted reads, SDK batching where safe, shorter critical sections.
Photo backlog	40-photo batches were read while uploads were deferred during DNS outage.	Durable/resumable face queue.
Result delivery ambiguity	CREATE_USER logged 'ok' followed by backend DNS delivery error.	Separate device execution result from result-delivery status.
409 concurrent modification	Backend REST command polling returned CONCURRENT_MODIFICATION.	Version-aware command protocol.
Unexpected gateway termination	BackendLink DisposeAsync NullReferenceException observed.	Null-safe lifecycle/disposal.

Priority	Work
P0	Door/safety-critical commands
P1	User create/update/delete/disable
P2	Event-driven targeted read

Priority	Work
P3	Roster reconciliation
P4	Face sweep/bulk import

Recovery	Expected behavior
Gateway restart	Load ProgramData config, roster state and outbox; reconnect; resume pending messages/photos; reconcile safely.
Device restart	Reconnect; do not infer deletion from first suspicious scan.
Device reset	Require explicit admin reset/re-enrollment policy; do not infer intent from a small roster.
Backend recovery	Reconnect and drain durable changes/photos in controlled batches.
Partial photo import	Resume remaining users; successful uploads remain acknowledged.
Failed individual photo	Retry that user; do not lose unrelated successful users.

Scenario	Expected state	Invariant
Backend online + device online	Normal bidirectional sync	Immediate event + durable ack
Backend offline + device online	Device remains usable	Queue outbound durably
DNS unavailable	Backend unavailable	Backoff; preserve queue
WebSocket drops	Reconnect/fallback	No message loss
REST 409	Conflict	Reload/reconcile
Gateway restart	Resume	Load outbox/state
Device restart	Reconnect	No mass delete
Empty roster	Failed read	No deletion
Partial roster	Failed read	No deletion
Direct name edit	Device event	Targeted report
Direct face edit	Device event	Hash + upload
Direct create	Device event	Create/link
Direct delete	Device event	Only trusted deletion
Server update	Server command	Version-aware apply
Both devices change	Conflict	Deterministic backend policy
Photo read failure	Partial sync	Retry user
Photo upload failure	Transport failure	Durable retry
One batch item fails	Partial batch	Ack successes; retry failure
1196-user initial import	Bulk sync	All users, bounded batches
Large face sweep	Background	Yield to priority work

Scenario	Expected state	Invariant
Two-device bulk sync	Parallel edge work	Per-device executors
Roster file locked	Persistence failure	Single writer/retry
Directory missing	Persistence failure	Create/repair
SDK succeeds; backend fails	Two-stage result	DEVICE_SUCCESS + DELIVERY_DEFERRED
Duplicate command	At-least-once delivery	Idempotent processing
Stale command	Version conflict	Ignore/reconcile

Priority	Change	Reason
P0	Remove ChannelReader.Count usage	Observed production exception breaks command acceptance.
P0	Make RosterStateStore single-writer and atomic	Observed file-in-use/access-denied/directory errors.
P0	Preserve incomplete-roster protection	Prevents mass deletion.
P0	Separate device success from delivery failure	Prevents misleading command status.
P1	Backend circuit/backoff	DNS/WebSocket failures caused repeated retries.
P1	Per-device priority executor	Bulk work delays commands.
P1	Durable resumable photo queue	Photos are large and outages occur.
P1	Face hash/metadata comparison	Avoid unnecessary photo transfers.
P2	Versioned backend conflict resolution	Required for two-device changes.
P2	Batch face endpoint	Reduce HTTP/DB overhead.
P3	Cursor-based attendance reconciliation	Avoid repeated broad scans.

› **Recommended final architecture: asynchronous durable backend transport → per-device priority executor → TrueFace SDK, with event-driven targeted reads and periodic reconciliation as safety mechanisms.**